

CASE CLOSED

Unity build log — architecture, decisions, and the bugs that shaped them

Repository: github.com/Brawly0/COURT · Unity 6000.5.6f1 · Netcode for GameObjects 2.13.1

Contents

- 1. What the game is
- 2. Tooling — the Unity/Claude bridge
- 3. Milestone 1 — Third-person player prototype
- 4. Milestone 2 — Multiplayer foundation
- 5. Milestone 3 — Proximity voice chat
- 6. Milestone 4 — Voice occlusion + secrecy fix
- 7. Milestone 5 — Greybox courthouse
- 8. Milestone 6 — Case generator integration
- 9. Milestone 7 — Role assignment
- 10. Milestone 8 — Briefings and ready flow
- 11. Milestone 9 — Networked interaction system
- 12. Milestone 10 — Archive search and evidence discovery
- 13. Milestone 11 — Physical evidence custody
- 14. Architecture summary
- 15. Every bug found, and what it taught
- 16. Known limitations
- 17. How to run and test
- 18. Recommended next steps

1. What the game is

CASE CLOSED is a 4–8 player courtroom-detective game. A procedurally generated crime, a 15-minute investigation, an 8-minute trial before a deterministic AI judge, and a truth reveal at the end.

The defining constraint, which drives nearly every decision in this log: **the game is about asymmetric knowledge**. The defendant knows whether they did it. Nobody else does. Players reconstruct the truth by physically moving around a building, searching for records, and talking to each other. That means secrecy is not a feature bolted on later — it is the thing the architecture has to protect at every layer.

Pre-existing work (not built in this log)

Component	State
<code>src/CaseClosed.TruthEngine</code>	Pure C# case generator. netstandard2.1, zero dependencies, own PCG32 RNG. 18 tests.
<code>Assets/Plugins/CaseClosed.TruthEngine.dll</code>	Compiled engine, already dropped into Unity.
<code>docs/MAP_DESIGN.md</code>	Level design doc defining the run structure and roles.

Why this matters: the generator answers "*what happened?*" Unity answers "*where can players find out?*" These were kept strictly separate throughout. The generator was never modified.

2. Tooling — the Unity/Claude bridge

Work began by connecting Claude Code to the Unity Editor so changes could be made and verified without manual round-trips.

Two bridges ended up available: Unity's own local relay (`unity` MCP), and the third-party **AI Game Developer** plugin (IvanMurzak/Unity-MCP) from a YouTube tutorial.

Decision: the plugin defaulted to **Cloud** mode, routing scene data through `ai-game.dev` and requiring an OAuth login. Switched it to **Custom** mode, which runs `gamedev-mcp-server.exe` on `localhost:24718` — no third-party routing, no account. The plugin's config had to be flipped on disk and a domain reload forced before it took effect.

3. Milestone 1 — Third-person player prototype

GOAL: spawn a character, walk, run, sprint, jump, camera follows, animations work

What existed, and what was left alone

The project already had `FirstPersonController.cs` wired into a networked player prefab, with the old courthouse calibrated against its speeds. **It was not modified.** The new third-person system was built alongside it.

Components created

File	Responsibility
<code>PlayerInputReader</code>	The <i>only</i> script that reads a keyboard or mouse.
<code>PlayerMovement</code>	Speed, gravity, slopes, jump, ground detection, turning.
<code>PlayerCameraRig</code>	Third-person orbit camera with collision.
<code>PlayerAnimatorDriver</code>	Reads movement, writes Animator parameters.
<code>PlayerDebugHud</code>	F1 overlay: speed, grounded, state.

Why input is isolated: when multiplayer arrives, a remote player's inputs get written into the same fields from the network instead of from a keyboard, and `PlayerMovement` never has to change. This one decision made Milestone 2 straightforward.

Character and animation

A blocky low-poly human built entirely from Unity primitives in an editor script — head, torso, arms, legs, hands, feet, simple face, suit. Seven animation clips (`Idle` , `Walk` , `Run` , `Sprint` , `Jump` , `Fall` , `Land`) generated procedurally as keyframed limb rotations, driven by a blend tree on measured velocity. No imported art.

Measured results

```
walk / run / sprint : 1.90 / 4.30 / 7.00 m/s
jump height       : 1.38 m against a 1.40 m target
ramps             : 25 deg walkable, 60 deg slides
leg swing (running) : -41.9 deg .. +42.0 deg (clip authored at 42)
arm swing          : -37.9 deg .. +38.0 deg (clip authored at 38)
hip bob            : 0.040 m (clip authored at 0.040)
```

4. Milestone 2 — Multiplayer foundation

GOAL: two players connect, see each other move, jump, animate; handle disconnects

Technology

Netcode for GameObjects 2.13.1, already in the project. Owner-authoritative: each player simulates their own character and broadcasts the result.

What runs where

Machine	Runs
Host	Server + a player. Relays traffic, owns spawning.
Your own character	Input, movement, camera, animation.
Someone else's character	Nothing but presentation — a puppet moved by network data.

Reused hard-won knowledge: a comment in the project's existing `NetPlayer.cs` read "*a live CharacterController stomps external transform writes, freezing remote copies at spawn.*" That was from an earlier "Fix frozen remote players" commit. Remote characters therefore get their `CharacterController` **disabled**.

Animation replication

Animations are **not** sent as clips. Four small values cross the wire — speed, vertical speed, grounded, movement state — and every machine runs the same Animator Controller off them.

Why: a few bytes per tick instead of a stream of animation state, and remote characters blend Idle→Walk→Run→Sprint using the identical state machine, so they cannot drift out of sync with what the body is doing.

Verified with two real processes

```
[Roles] host and client each hold 2 players; each owns exactly one
host tracked client z -3.1 -> -34.2 ; client tracked host z 5.3 -> 36.0
remote states seen: Walk, Run, Sprint, Jump, Fall, Land
remote leg swing 111.4 deg (Sprint clip authored at 56 => 112)
disconnect: "client 1 left", host continued; 0 exceptions
```

5. Milestone 3 — Proximity voice chat

GOAL: hear nearby players, quieter with distance, silent beyond range

Technology decision

Built on **NGO itself**, not Vivox.

Why not Vivox (the usual answer): it requires a Unity Gaming Services project, dashboard linking and credentials — account setup that would have blocked the milestone entirely, plus a second network stack alongside NGO. The cost of not using it is real and was stated up front: no echo cancellation, no noise suppression, no AGC.

The audio path

```
Microphone -> 16 kHz mono -> G.711 mu-law -> 20 ms frames (320 bytes)
-> ServerRpc -> host measures distance per listener
-> ClientRpc to those in range only
-> jitter buffer -> streaming AudioClip -> AudioSource ON THE SPEAKER'S BODY
```

Codec measured SNR	36.6 dB (telephone quality is ~38 dB)
Compression	4:1 versus float PCM
Bitrate	~128 kbit/s per active speaker

Two-layer attenuation. The *server* drops frames for listeners beyond max range — that is the hard cutoff, and means a modified client cannot listen in from across the map because those packets were never sent. Within range, the AudioSource sits on the speaker's character at `spatialBlend = 1`, so distance and direction come free.

Verified across processes

```
dist= 8.2m  audioArriving=True  level=0.248
dist=14.8m  audioArriving=True  level=0.250
dist=28.2m  audioArriving=False level=0.000
dist=72.2m  audioArriving=False level=0.000
```

6. Milestone 4 — Voice occlusion + secrecy fix

GOAL: walls should block voice

Distance alone meant a player two metres away through a sealed courtroom wall was as loud as one standing next to you. For COURT that is worse than wrong — "*can they hear me from in there*" is the actual game.

Occlusion, two layers again

- **Server (authoritative):** walls shrink how far a voice carries. A modified client cannot hear through a wall, because those packets are never sent.
- **Client (presentation):** volume drop *plus* a low-pass filter.

Why the low-pass matters: volume alone just sounds far away. Losing the high frequencies is what reads as "through a door". Interpolated in log space, because perceived brightness is logarithmic, and smoothed over 0.18 s so doorways fade rather than pop.

open floor, clear line	blockers=0	occlusion=0.00	carries 21.0 m
through one corridor wall	blockers=1	occlusion=0.33	carries 16.5 m
through both corridor walls	blockers=2	occlusion=0.67	carries 11.9 m

One wall *muffles* rather than silences — deliberately, since overhearing a muffled argument through a courtroom door is a mechanic worth having.

Leak found and closed. `IsSpeaking` was a `NetworkVariable` readable by everyone, so a client knew a player across the map was talking even though the server had correctly refused them the audio. In a game built on eavesdropping that is exactly the information you must not leak. It is now **derived from audio actually received** — zero bandwidth, and it cannot lie.

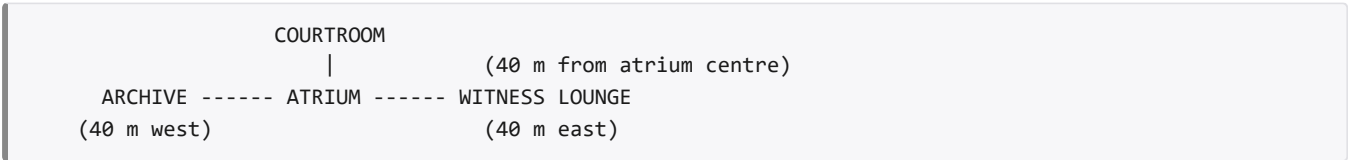
Before: `dist=72.2m speaking=True` → After: `dist=72.1m speaking=False`

7. Milestone 5 — Greybox courthouse

GOAL: Archive, Witness Lounge, Courtroom around a central Atrium

The previous v2.3 courthouse was removed at the user's request (all recoverable from commit `c272023`), along with the first-person player and the scripts welded to it.

The plan is a hub and three spokes



Every room is 40 m from the middle — about 9.3 s at the controller's 4.3 m/s run speed, inside the 8–12 s target. Any two rooms are 80 m apart via the atrium (~18.6 s), under the 20 s ceiling.

Everything routes through the atrium on purpose. There are no shortcuts between wings, so crossing the building means passing through the one space where everyone else is. That is where chance encounters and overheard conversations happen. A ring corridor would be kinder to navigate and would kill the game.

Room	Contents and intent
Archive	Shelf aisles chop it into lanes, so two players searching can be metres apart and neither see nor clearly hear each other.
Witness Lounge	Open waiting area plus an interview nook behind a partition — the first place you can hold a conversation others cannot quite hear.
Courtroom	Judge dais, witness stand, opposed counsel tables, bar, gallery benches. Positions only, no trial logic.
Atrium	Pillars break sightlines; reception desk as a landmark.

`RoomVolume` + `GreyboxDebugHud` (F3) record real travel times during play rather than calculating them from distances.

8. Milestone 6 — Case generator integration

GOAL: host generates a case, owns the truth, clients get only what they may know

Correction made during this milestone. The generator was initially wired in as a local package. That was wrong — a current, git-tracked `CaseClosed.TruthEngine.dll` already sat in `Assets/Plugins` . The compiler caught it (`CS1704: assembly already imported`); the package was reverted and the existing DLL used.

The four data layers

Type	Holds	Who sees it
<code>CompleteCaseTruth</code>	Everything: perpetrator, occupancy, corruptions, proof chain	Host only
<code>PublicCaseInfo</code>	Title, crime, briefing, cast, duration	Everyone
<code>PlayerCaseView</code>	Role, private briefing, permitted facts, guilt (defendant only)	One player

DeveloperCaseDebugView	Everything, formatted	Host, dev builds, behind F2
------------------------	-----------------------	-----------------------------

Protection is structural, not procedural. `CompleteCaseTruth` is a managed class that does *not* implement `INetworkSerializable` and contains dictionaries and jagged arrays — so `SomethingClientRpc(truth)` is a **compile error**. `ActiveCaseManager` has no `NetworkObject`, so it cannot replicate even by accident. There is no permission to misconfigure.

The guilt bit is sent as `false` to non-defendants *regardless of the truth*, so the packet carries no signal either way.

Bug caught by the audit: `FixedString128Bytes` is sized in **bytes**, but the truncation helper counted **characters** — and the generator's prose contains em-dashes (3 UTF-8 bytes each). A 124-character string could be 130 bytes and throw. Fixed to a byte budget.

9. Milestone 7 — Role assignment

GOAL: server-authoritative roles, exactly one Defendant, safe late join

Roles came from the project's own `docs/MAP_DESIGN.md` rather than being invented:

LOBBY Roles dealt. Defendant learns the truth, alone, on his card.
BRIEFING Both teams enter their own office. Prosecution gets 1 forensic fact. Defense gets nothing and has to ask their client.

Role	Team	Unique?
Defendant	Defense	Yes — dealt once per case, never re-dealt
Defense Attorney	Defense	Yes
Prosecutor	Prosecution	Yes
Investigator	Prosecution	No — repeatable seat

The judge is the deterministic AI, never a player.

Roles are public; guilt is not. In a courtroom you can see who is prosecuting. Hiding it would buy no secrecy and would make name tags and office doors impossible. Guilt travels on a separate channel to one player. Keeping them apart stops a future feature widening the roster and leaking the mystery.

A departed Defendant is not replaced. Promoting someone mid-case would hand a stranger the answer to the mystery. The seat goes vacant, `DefendantMissing` flips true, and the trial system decides between in-absentia (which the design doc already names) and a redeal.

Dealt from the case seed using the project's own PCG32, with client ids **sorted before shuffling** — NGO reports connections in arbitrary order, so without sorting "deterministic" would be a lie.

10. Milestone 8 — Briefings and ready flow

GOAL: every player gets role, team, public info, private briefing, and a Ready button

A match-level state machine kept separate from the case lifecycle:

```
LobbyReady -> AssigningRoles -> GeneratingCase -> DistributingBriefings  
            -> WaitingForPlayers -> PreInvestigationReady
```

Why separate: `CaseLifecycleState` answers "does a generated case exist and has it been distributed" — a data question. `MatchPhase` answers "what are the humans doing" — a session question. Merging them would mean regenerating a case rewinds the lobby.

Readiness

`SubmitReadyServerRpc` takes **no client id** — NGO supplies the sender, so marking another player ready is *unrepresentable* rather than merely validated. A server-side `HashSet` makes duplicates idempotent; disconnect withdraws readiness.

A real leak, found by strengthening a weak test. The first "hides the perpetrator" check ran on a *guilty* case — where the perpetrator **is** the defendant, so it passed vacuously. Re-running against an *innocent* case exposed that the Prosecutor's opening fact was `ProofFacts[0]` — and the proof chain is precisely what implicates the real culprit. The Prosecutor was being handed the answer at the briefing screen.

Fix: sourced from the defendant's **baggage** instead — true, discoverable, about the defendant, and never naming the culprit. Better design, not just a patch: baggage is *why the state charged them*.

Verified with four processes

```
[Roles] Dealt - 1 Defendant, 1 Defense Attorney, 1 Prosecutor, 1 Investigator
phase: AssigningRoles -> GeneratingCase -> DistributingBriefings
      -> WaitingForPlayers -> PreInvestigationReady
Ready 1/4 -> 4/4 -> ALL PLAYERS READY

p0 DefenseAttorney/Defense    knowsGuilt=False guiltBit=False
p1 Investigator/Prosecution  knowsGuilt=False guiltBit=False
p2 Defendant/Defense          knowsGuilt=True  guiltBit=True
p3 Prosecutor/Prosecution     knowsGuilt=False guiltBit=False

truthDigest appears 11x in the host log, ZERO times in all three client logs
```

11. Milestone 9 — Networked interaction system

GOAL: one reusable interaction contract for shelves, doors, terminals, the gavel

The contract

`NetworkInteractable` is an abstract base, not an interface.

Why a base class: the parts that must not vary — distance checks, the exclusivity lock, hold timing, release on disconnect — live in one place and cannot be forgotten by an implementer. An interface would let every future object re-invent locking, and one of them would get it wrong.

What a client may say

"I would like to interact with object 47." That is the entire vocabulary. No position, no result, no completion time. The server resolves the id through NGO's spawn table — a fabricated id resolves to nothing — measures distance itself, and runs its own clock.

Server validation order (cheapest first)

1. Player exists and is spawned
2. Player state allows interaction
3. Target is available
4. Not locked by another player
5. Role/team permission
6. Distance (measured server-side)
7. Line of sight
8. The object's own `ServerValidate`

Holds tick in the *server's* `Update`, re-running full validation every frame. Walking away, losing sight, or the object becoming unavailable cancels without the client's cooperation. The client is never asked how long it held the key.

Bug 1 — Unity script binding. Four `MonoBehaviours` were put in one file. The project already documents this in `ClientNetworkTransform.cs`: *"Unity's script-to-prefab binding breaks for MonoBehaviours defined in a file named after a different class."* Split into separate files.

Bug 2 — stuck asset import. `InteractionNetworkController.cs` was valid C#, had a unique GUID, and Unity refused to emit its type *while reporting no error against the file*. Forced reimport and recompile did not help. **Deleting the asset and recreating it with a fresh GUID did.** Worth remembering: if a type "doesn't exist" with no error against its file, that is the fix.

Bug 3 — the one that mattered: raycasting from the camera. COURT is *third-person*. The camera sits ~5 m behind the character (measured), and the probe range was 3.5 m — so the ray expired about a metre behind the player's own back and could never reach anything in front of them. Nothing was ever targetable; pressing E did nothing.

Fix: cast from the **player's eyes, along the camera's facing**. You reach for what you are looking at, from where you actually are. It also matches how the server measures distance, so the prompt cannot promise something the server then refuses.

Bug 4 — stale prefab. The interaction detector was added to the prefab *builder*, but only the *scene* builder had been re-run. The prefab was stale and the component simply was not on the player. Diagnostic output: detector on player : False .

Feedback added after "I can't tell if E did anything"

The crosshair became the confirmation: grows amber when something is targetable, punches white on press *before the server answers*, then flashes green on success or red on refusal. Plus a message when no session exists — that silence was indistinguishable from a bug.

12. Milestone 10 — Archive search and evidence discovery

GOAL: search Archive containers, discover real generated evidence, reach FOUND

Key inspection finding

The generator already answers the placement question. Every `EvidenceItem` carries a `FoundAt` string:

#	Item	FoundAt	Archive?
0	{CrimeObject} (recovered)	Impound cage, parking garage	no
1	Fingerprint card: ...	Lab tray (90s)	no
2	CCTV tape: ...	Security office console	no
3	Catering schedule	Archives, row 3	yes
4	Door log: {location}	Archives, row 5	yes
5	Toxicology slip (Fractured only)	Lab tray (120s)	no

So categories were not invented. Unity reads the generator's own intent, which is what keeps fingerprints and stolen goats out of filing cabinets. Three gaps were *flagged rather than smuggled in*: no stable `EvidenceId` (derived as `E-003` from list position), no type enum (parsed from `FoundAt`), and only 2 Archive-suitable items per case.

Secrecy of placement

The placement map is a plain `Dictionary` on `ArchiveDirector` — never a `NetworkVariable`, never an RPC parameter. `ArchiveContainer` has **no contents field at all**; it cannot leak what it does not know, and asks the director at completion.

Ten containers

Desk Drawer	1.5 s
Records Box	2.5 s
Filing Cabinet	3.0 s
Shelf Section	4.0 s

Junk is deliberately inert — no character names, no times, no locations. It must waste a player's time, never mislead them into an accusation the case cannot support.

Placement seeds from `caseSeed XOR salt` on a **separate PCG32 stream** from the case and the role deal, so reshuffling the Archive can never disturb either.

13. Milestone 11 — Physical evidence custody

GOAL: FOUND -> CARRIED -> DROPPED, and another player can pick it up

The central question, answered

The audit found the existing `FOUND` was **knowledge**, not custody — it recorded *who learned it* and had no notion of a physical object. So the two were split:

	Shape	Behaviour
Knowledge	a set of readers per item	only ever grows
Custody	a single field per item	exactly one location

That asymmetry does the work. Knowledge is a set, so several people can know the same thing and nobody forgets when they hand a folder over. Custody is one field, so **duplication is unrepresentable** — an item cannot be in a drawer *and* a hand, because there is nowhere to write the second location.

The loop

Discovery reveals a **manila folder into the world**, not into an invisible inventory — you know what the document says before you hold it. Carry limit is one. Carrying is not physics: the server owns *who holds it*, and every client draws it at that player's socket.

Drop (`G`) takes **no position from the client** — a client-chosen position could post evidence through a wall. Inspection (`Tab`) never shrinks: drop a folder and you still remember what it said.

On disconnect the carrier's item drops where they last stood and stays in play. Deleting it would silently remove a clue; returning it to its drawer would let a player hide evidence by pulling their network cable.

Bug — stale bodies. Rebuilding placement reset the evidence records but left folders from the previous case lying on the floor, still carrying ids that had just been reset — world objects with no backing record. Custody is now cleared whenever knowledge is.

Bug — evidence spawning into walls. Folders were revealed along `container.transform.forward`, but containers are built unrotated, so every forward is world +Z. The north row spawned its evidence at $z=9.5$, half a metre from the wall at $z=10$ — unreachable. Containers now carry an explicit `RevealDirection`.

Final verified run

```
evidence E-003 in container 3
after search : knowledge=Found custody=InWorld
folder at    : (-36.0, 1.3, 7.5)
aimed at     : PhysicalEvidence
after pickup : custody=Carried carrier=0
HUD          : True "Catering schedule (places staff by slot)"
2nd carrier? : refused
after drop   : custody=InWorld
knowledge    : kept=True readers=1

VERDICT      : FOUND -> CARRIED -> DROPPED, KNOWLEDGE KEPT, NO DUPLICATION
```

14. Architecture summary

Truth	CompleteCaseTruth	host memory only, not serializable
Knowledge	EvidenceInstance.KnownBy	a set; only ever grows
Custody	EvidenceInstance.Custody	one field; one location
Registered	(reserved, not built)	
Admissible	(reserved, not built)	

Layer responsibilities

Layer	Files	Owns
Player	Scripts/Player/Prototype/	Input, movement, camera, animation
Multiplayer	.../Prototype/Net/	Ownership gating, transform + animation replication
Voice	.../Prototype/Voice/	Capture, codec, occlusion, spatial playback
Greybox	Scripts/Greybox/ , Editor/Greybox/	Rooms, room volumes, travel timing
Case	Scripts/Cases/	Generation, secrecy boundary, view filtering
Roles	Scripts/Cases/Roles/	Deal, roster, teams
Match	Scripts/Match/	Phases, briefings, readiness
Interaction	Scripts/Interaction/	The one interaction contract
Archive	Scripts/Archive/	Placement, search, discovery, custody

Automated audits (all passing)

Audit	Checks	Menu
Case integration	41	Case Closed > Cases > Run Integration Audit
Interaction	21	Case Closed > Interaction > Run Audit
Archive + custody	36	Case Closed > Archive > Run Audit

Why many checks are structural (reflection) rather than behavioural: "a client cannot impersonate another player" is a property of an RPC signature, not of any particular playthrough. Asserting it against the shape of the code catches a regression the moment someone adds a convenient `clientId` parameter — which no amount of play-testing reliably would.

15. Every bug found, and what it taught

#	Bug	Lesson
1	Animator controller assigned to the scene instance, never the prefab — so nothing animated after switching to network spawning	Verify the <i>observable effect</i> (limbs moving), not the proxy (state names)
2	<code>PlayerCameraRig.Start()</code> dereferenced a null Target once the player spawned later	Spawn order changes what exists at Start
3	<code>FixedString128Bytes</code> counts bytes; truncation counted characters; em-dashes are 3 bytes	Fixed-size network strings are byte budgets
4	Prosecutor's opening fact came from the proof chain — handing them the culprit	A test that passes vacuously is worse than no test
5	<code>IsSpeaking</code> replicated globally, revealing distant talkers	Derive from what a player legitimately received
6	Four MonoBehaviours in one file broke Unity script binding	The project had already documented this exact trap
7	Unity refused to compile a valid file with no error; fresh GUID fixed it	Delete and recreate when a type "doesn't exist" silently
8	Interaction raycast originated at the camera — 5 m behind the player in third person	First-person instincts do not transfer
9	Prefab builder updated but never re-run; component missing at runtime	Rebuild the artefact you changed, not just the scene
10	Placement rebuild left stale evidence bodies in the world	Clear custody whenever knowledge is cleared
11	Evidence revealed along <code>transform.forward</code> spawned inside walls	Unrotated objects all share world forward
12	<code>NetworkConfig</code> intermittently null after <code>AddComponent</code> in edit mode	Guarantee it rather than depend on timing

Testing mistakes worth recording

- **A verdict reported PASS while the pickup never happened** — the condition only checked state that was already true beforehand. Verdicts must require the transition, not the end state.
- **Hard-killing a process is not a disconnect** — it sends nothing, so NGO waits out its ~30 s transport timeout. A graceful leave was needed to test the path. (This is also real production behaviour: a crashed player lingers ~30 s.)
- **Aiming the player is not aiming the camera** — the camera rig keeps its own yaw, so tests had to place the player relative to where the camera actually looked.

16. Known limitations

Area	Limitation
Movement	Owner-authoritative — a modified client could teleport or fly. Fine for prototype.
Networking	Direct IP only. No NAT traversal until the Steam relay. No host migration.
Networking	No client-side prediction; remote players are ~1 tick in the past.
Networking	Client ids are reassigned on reconnect — a returning player is a stranger to the roster.

Voice	No echo cancellation, noise suppression or AGC. Headphones required.
Voice	~128 kbit/s per speaker; Opus would cut that ~8x but needs a native plugin.
Voice	Occlusion is a single ray with no notion of an open door.
Roles	Verified at 4 players; spawn slots handle 6 but are untested.
Briefing	Late join after PreInvestigationReady does not rewind the phase.
Archive	Only 2 Archive-suitable items per case , so 8 of 10 containers are junk or empty.
Evidence	Discovery is individual; no team sharing. Players must talk. Deliberately undecided.
Evidence	Handoff not implemented — dropping is the transfer mechanism.
Evidence	<code>EvidenceId</code> is positional; changing generation order shifts ids.
Build	No .NET SDK on the machine — the engine's 18 tests cannot run and the DLL cannot be rebuilt.

17. How to run and test

From the editor

Open `Assets/Scenes/Courthouse.unity`, press Play, click **HOST**.

Multiplayer

Build via *Case Closed* > *Prototype* > *4. Build Playground Player*, then launch

`game/Build/Prototype/PlayerPrototype.exe` two to four times. First window HOST, the rest JOIN 127.0.0.1.

Controls

WASD / mouse	move / look
Shift / Ctrl	sprint / walk
Space / R	jump / respawn
E	interact (hold for searches)
G	drop carried evidence
V	push to talk
M / N	mute / next microphone
Tab	inspect evidence you know
F1 / F2 / F3 / F5	movement / case debug / greybox / role

The full loop

1. HOST, then F2 → **START MATCH** (deals roles, generates case, sends briefings)
2. Each player reads their briefing and presses **READY**
3. Walk west to the **ARCHIVE**
4. Hold `E` on containers — most are empty or junk; two hold real evidence
5. On a find, a manila folder appears. Press `E` to pick it up
6. `G` to drop; another player can take it. `Tab` still shows what you read

18. Recommended next steps

1. **Evidence Locker and registration** — `CARRIED` now has somewhere to go, and `EvidenceCustody.Registered` already has its number reserved.
2. **Playtest before expanding the generator.** The 2-items-per-case sparseness is a playtesting question, not a bug to pre-solve.
3. **Persistent player identity** — needed before reconnection can work.
4. **Server authority on movement** — before anything competitive.
5. **Opus voice codec** — before player counts rise.
6. **Install the .NET SDK** so the engine's 18 tests can run again.

Generated from the full build session. Every measurement quoted was produced by a verification run, not estimated. All three audits passing at time of writing: case 41/41, interaction 21/21, archive + custody 36/36.